

Hochschule Bremen
City University of Applied Sciences



Fallstudie Bürgerregister Light

Bilal Özsoy, Felix Hinrichs, Jan Plachetka, Nick Eilers

Agenda

1. Anwendung & Ausgangslage
2. Qualitätsprobleme
3. Clean-Code-Prinzipien
4. Code Smells (+ Lösungen)
5. Teststrategie und Tools
6. Reflexion & Fazit

Anwendung & Ausgangslage

Zweck und Scope der Anwendung

"Im Rahmen des Moduls Software Engineering II soll daher ein vereinfachtes, aber realitätsnahes Fallbeispiel entwickelt werden: das „Bürgerregister Light“."

Module/Komponenten

Erfassung von Bürgerdaten - Validierung - Speicherung - Abfrage und Anzeige

Ausgangszustand der Codebasis

- Code muss refactored werden
- Offensichtliche Fehler im Code
- dennoch: funktionales Register mit GUI

Qualitätsproblem 1

Single-Responsibility-Verstoß (SRP): Parsing/Validierung im Register

Die Klasse Buergerregister verwaltet Personen – enthält aber zusätzlich Logik zum **Parsen + Validieren von Geburtsjahren** (parse_birth_year). Das ist fachlich eher **Input-/Validierungslogik** als Register-Logik.

Warum smell?

- Vermischung von Verantwortlichkeiten → schlechter testbar, wächst schnell aus (weitere Parsing-Funktionen landen dann auch “irgendwo”).

Refactoring-Idee:

- parse_birth_year in ein Modul wie validation.py oder einen BirthYearParser/Validator auslagern (oder direkt in validiere_person integrieren).

Qualitätsproblem 1

```
def find(self, nachname: str) -> List[Person]:
    """Sucht Personen anhand des Nachnamens (Groß-/Kleinschreibung wird beachtet)."""
    # List Comprehension für Filterung
    return [p for p in self._personen if p.nachname == nachname]

def parse_birth_year(self, year_str: str) -> int:
    """
        Parses a birth year from a string, validating it to be between 1900 and 2025.

        Args:
            year_str (str): The year string to parse.

        Returns:
            int: The parsed year as an integer.

        Raises:
            ValueError: If the year is not a valid integer or out of range.
    """
    try:
        year = int(year_str.strip())
```

Qualitätsproblem 2

Magic Numbers / Hardcoded Range (1900–2025)

In `parse_birth_year` ist der gültige Bereich **fest verdrahtet** ($1900 \leq \text{year} \leq 2025$). Das ist ein typisches “Magic Number”-Problem.

Warum smell?

- 2025 wird sofort “alt” (2026+), erzeugt Wartungsaufwand und potenzielle Bugs.

Refactoring-Idee:

- Konstanten einführen (MIN_YEAR, MAX_YEAR) und MAX_YEAR dynamisch machen (z. B. aktuelles Jahr), oder Regeln zentral in validation definieren.

Qualitätsproblem 2

```
    """  
    try:  
        year = int(year_str.strip())  
    except ValueError:  
        raise ValueError("Invalid year format")  
    if not (1900 <= year <= 2025):  
        raise ValueError(f"Year {year} out of range (1900-2025)")  
    return year
```

Qualitätsproblem 3

Duplikatprüfung über “Vorname+Nachname”

Duplikate werden über `any(p.nachname == person.nachname and p.vorname == person.vorname ...)` geprüft.

Warum smell?

- Fachlich fragil: Viele Menschen teilen Vor- und Nachnamen → **False Positives**.
- Technisch: Die Logik ist “verstreut” statt über einen eindeutigen Schlüssel/Identität modelliert.

Refactoring-Idee:

- Eindeutigen Identifier in Person (z. B. `person_id`) oder eine definierte Identitätsregel (z. B. zusätzlich Geburtsdatum/Adresse).
- Optional: Intern eine dict/set-Struktur nach Schlüssel statt linearem Durchsuchen (skalierbarer, klarer).

Qualitätsproblem 3

```

if not ok:
    # Fehlerliste zu einem String zusammenbauen
    return False, "Validierungsfehler: " + ", ".join(errors)

# 2. Duplikatprüfung (Vorname + Nachname)
# Wir nutzen einen Generator-Ausdruck für Effizienz
if any(p.nachname == person.nachname and p.vorname == person.vorname for p in self._personen):
    return False, "Warnung: Diese Person existiert bereits (Duplikat)."
```

```

# 3. Speichern
self._personen.append(person)
return True, "Person erfolgreich hinzugefügt."
```

```

def list(self) -> List[Person]:
```

Clean-Code-Leitlinien 1

Meaningful Names

- Verständlichkeit von Variablen und Funktionen sind jederzeit gegeben

•	<code>vorname: str</code>	<code>X name</code>
•	<code>nachname: str</code>	<code>X name</code>
•	<code>geburtsjahr: int</code>	<code>X jahr</code>
•	<code>wohntort: str</code>	<code>X Ort</code>

SRP - Single Responsibility Principle (dt. Ein-Aufgaben-Prinzip)

- Projekt ist so aufgeteilt, das jede Datei genau eine Verantwortung übernimmt
- besserer Wartbarkeit, Verständlichkeit und Testbarkeit der Software
- `models.py` -> **Datenstruktur**
- `validation.py` -> **Logik-Prüfung**
- `register.py` -> **Verwaltung (Hinzufügen, Suchen, Auflisten)**

Clean-Code-Leitlinien 2

Fail Fast (dt. schnelles Scheitern)

- Ein Fehler soll so früh wie möglich gemeldet werden, anstatt mit ungültigen Daten zu arbeiten
- Vorteil: Keine Verschmutzung der Daten -> keine Verschwendung von Rechenpower

```
# 1. Validierung aufrufen (Umwandlung in Dict für den Validator)
ok, errors = validiere_person(person.__dict__)
if not ok:
    # Fehlerliste zu einem String zusammenbauen
    return False, "Validierungsfehler: " + ", ".join(errors)

# 2. Duplikatprüfung (Vorname + Nachname)
# Wir nutzen einen Generator-Ausdruck für Effizienz
if any(p.nachname == person.nachname and p.vorname == person.vorname for p in self._personen):
    return False, "Warnung: Diese Person existiert bereits (Duplikat)."
```

```
# 3. Speichern
self._personen.append(person)
return True, "Person erfolgreich hinzugefügt."
```

Code Smell 1: Einhaltung des SRP

Lösung: Auslagerung der parse_birth_year Funktion

1 Im "validation" File die neue Funktion Einfügen

```

def parse_and_validate_birth_year(year_str: str) -> int:
    """
    Parst ein Geburtsjahr und validiert den Bereich (1900-2025).
    Raises: ValueError bei ungültigen Format oder Bereich.
    """
    try:
        year = int(year_str.strip())
    except ValueError:
        raise ValueError(f"'{year_str}' ist keine gültige Zahl.")

    if not (1900 <= year <= 2025):
        raise ValueError(f"Das Jahr {year} liegt nicht zwischen 1900 und 2025.")

    return year
    
```

2 Im File "buergerregister.py": Funktion importieren und die alte Funktion rauslöschten

```

from .validation import
    parse_and_validate_birth_year
    
```


Wirkung des Refactoring:



Code Smell 2:

Lösung: Einführung einer Personen-ID als technischen Schlüssel

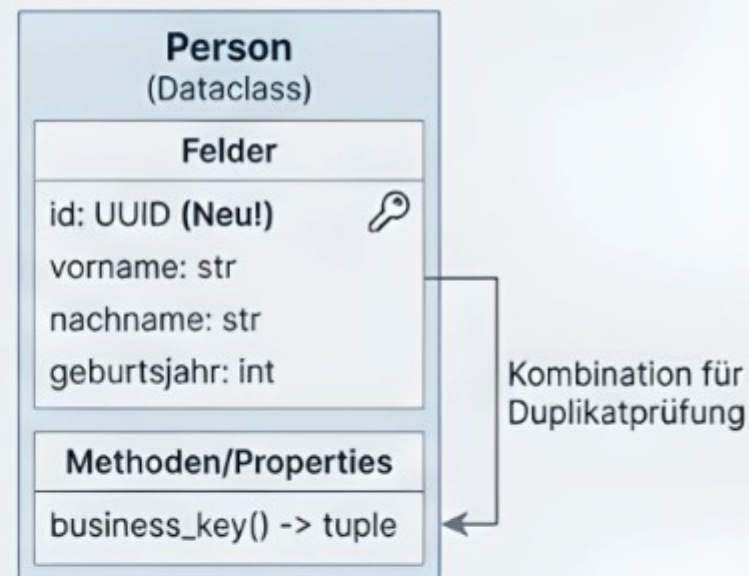
1: Anpassung der Klasse 'Person' -> Definition einer Fingerabdruck Methode

```
import uuid
from dataclasses import dataclass

@dataclass(frozen=True) # frozen=True macht die Klasse hashable für Sets/Dicts
class Person:
    id: uuid.UUID
    vorname: str
    nachname: str
    geburtsjahr: int # Hinzugefügt, um die Eindeutigkeit zu erhöhen

    @property
    def business_key(self):
        """
        Erzeugt einen fachlichen Schlüssel für die Duplikatprüfung.
        Kombination aus normierten Namen und Geburtsjahr.
        """
        return (self.nachname.lower(), self.vorname.lower(), self.geburtsjahr)
```

Visualisierung der Klasse Person



2: Umbau des Register files

```
class Buergerregister:
    def __init__(self):
        # Technischer Speicher: Zugriff über ID (schnell)
        self._personen_by_id: Dict[uuid.UUID, Person] = {}

        # Fachlicher Speicher: Verhindert Duplikate (schnell)
        # Speichert Tupel wie ('müller', 'Thomas', 1990)
        self._existing_identities: Set[tuple] = set()

    def add_person(self, person: Person):
        # 1. Schnelle Prüfung auf fachliches Duplikat (O(1))
        if person.business_key in self._existing_identities:
            raise ValueError(f"Person {person.vorname} {person.nachname} {{person.geburtsjahr}} existiert bereits.")

        # 2. Speichern
        self._personen_by_id[person.id] = person
        self._existing_identities.add(person.business_key)
        print(f"Person mit ID {person.id} hinzugefügt.")

    def find_by_id(self, person_id: uuid.UUID) -> Optional[Person]:
        return self._personen_by_id.get(person_id)

    def find_by_name(self, nachname: str) -> list[Person]:
        # Suche über Attribute bleibt linear, wenn kein Index existiert,
        # ist aber sauberer getrennt von der Identitätsprüfung.
        return [p for p in self._personen_by_id.values() if p.nachname == nachname]
```

Wirkung des Refactoring:

1. **Fachliche Korrektheit** -> Weniger "False Positives" und klare Definitionen
2. Performance und Skalierbarkeit: Höhere Geschwindigkeit und leichter Zugriff auf einzelne Personen
3. Erleichterte Wartbarkeit



Weniger "False Positives"
und klare Definitionen.



Höhere Geschwindigkeit
und leichter Zugriff auf
einzelne Personen.



Vereinfachte Pflege und
zukünftige Anpassungen.

Teststrategie zur Absicherung des Refactorings:

1. Isolation (Positiv-Test) ✓

Das Bürgerregister wird isoliert getestet.

```
import pytest
from buergerregister import Buergerregister, Person

def test_add_homonyms_with_different_birth_years():
    # Arrange: Initialisierung des Registers
    register = Buergerregister()
    id1 = uuid.uuid4()
    id2 = uuid.uuid4()

    # Zwei Personen: Gleicher Name, unterschiedliches Geburtsjahr
    person_a = Person(id=id1, vorname="Max", nachname="Müller",
    geburtsjahr=1990)
    person_b = Person(id=id2, vorname="Max", nachname="Müller",
    geburtsjahr=2000)
    # Act: Beide hinzufügen
    register.add_person(person_a)
    register.add_person(person_b)

    # Assert: Prüfung, ob beide im System sind
    # 1. Beide IDs müssen abrufbar sein
    gefunden_a = register.find_by_id(id1)
    gefunden_b = register.find_by_id(id2)
    assert gefunden_a == person_a
    assert gefunden_b == person_b

    # 2. Die Suche nach Namen muss nun 2 Ergebnisse liefern
```

2. Business-Rule Validation (Negativ-Test) ✗

Wir testen ob Duplikate verhindert werden (Namensvetter sollen erlaubt bleiben).

```
import uuid
import pytest
from buergerregister import Buergerregister, Person

def test_add_exact_duplicate_should_fail():
    # Arrange
    register = Buergerregister()
    # Eine Person existiert bereits
    original = Person(id=uuid.uuid4(), vorname="Lisa", nachname="Meier",
    geburtsjahr=1985)
    register.add_person(original)

    # Versuch, ein logisches Duplikat zu erstellen (neue ID, aber gleiche
    # Fachdaten)
    duplikat = Person(id=uuid.uuid4(), vorname="Lisa", nachname="Meier",
    geburtsjahr=1985)

    # Act & Assert: Wir erwarten, dass das System den Fehler wirft
    with pytest.raises(ValueError) as excinfo:
        register.add_person(duplikat)

    # Optional: Prüfen, ob die Fehlermeldung sinnvoll ist
    assert "existiert bereits" in str(excinfo.value)

    # Sicherstellen, dass das Register nicht korruptiert wurde (immer noch
    # nur 1 Eintrag)
    assert len(register.find_by_name("Meier")) == 1
```

Toolgestützte Codequalität

Eingesetzte Tools

Black – automatische, einheitliche Code-Formatierung

isort – konsistente Sortierung von Imports (kompatibel mit Black)

ruff – statische Codeanalyse / Linting

Nutzen / typische Befunde

Einheitlicher Stil im gesamten Projekt → bessere Lesbarkeit

Vermeidung typischer Fehler (z. B. ungenutzte Imports, Stilwarnungen)

Schnellere Reviews: Fokus auf Logik statt Formatierung

Hinweis

Tools verbessern die formale Qualität, ersetzen aber keine Tests oder Architekturentscheidungen.

Reflexion & Lessons Learned

Messbare Verbesserungen

- Einheitliche Formatierung → weniger „Diff-Rauschen“ in Git
- Schnellere Qualitätschecks über lint und Test
- Testabdeckung sichtbar durch Coverage-Report (HTML)

Grenzen & Trade-offs

- Tools erkennen keine fachlichen Logikfehler
- Regeln sind Kompromisse (z. B. Zeilenlänge vs. Lesbarkeit)
- Gute Tooling ersetzt nicht: Refactoring + saubere Tests

Learning

Tooling ist ein Sicherheitsnetz,
aber Qualität entsteht durch
Design + Tests.

Ausblick



Automatisierung in Github Actions:

Linting + Tests bei jedem Push



Testfälle Erweitern:

Mehr Rand-/Negativfälle
(Ungültige Eingaben,
Dubletten)



Type Hints ausbauen

Später optionale Typprüfung
(Z. B. mypy) Möglich

Fragen & Feedback

Hochschule Bremen
City University of Applied Sciences



Fallstudie Bürgerregister Light

Bilal Özsoy, Felix Hinrichs, Jan Plachetka, Nick Eilers

Danke für die Aufmerksamkeit

Hochschule Bremen
City University of Applied Sciences



Fallstudie Bürgerregister Light

Bilal Özsoy, Felix Hinrichs, Jan Plachetka, Nick Eilers